Fonaments d'Informàtica

5. Assemblador: l'ISA RISC-V

Enginyeria de Sistemes de Telecomunicació
Enginyeria Electrònica de Telecomunicació

Dpt. de Microelectrònica i Sistemes Electrònics
Curs 2026/27

UAB Universitat Autònoma de Barcelona

Índex

1. Introducció

2., 3., 4., 5. *Instruction Set*

6. Funcions, subrutines

7. Banc de registres

8. Pseudoinstruccions

9. Exemples

10. Codificació a codi màquina

1. Introducció: de la ISA del Tema 4 a RISC-V

Recordatori del Tema 4: la ISA és el contracte entre maquinari i programari. RISC-V és una ISA **RISC**, **load-store**, **oberta** i **modular** (Berkeley, 2010): base RV32I + extensions (M, F, D, C, V…).

- En aquest tema programarem el **model del programador de RV32I**:
  - 32 registres de 32 bits (x0 sempre val 0);
  - memòria adreçada per byte, little-endian, paraules de 32 bits alineades;
  - instruccions de 32 bits: aritmètiques i lògiques, accés a memòria i salts.
- Treballarem amb un **subconjunt de RV32I**: el que accepta el simulador de pràctiques (Cornell RISC-V web interpreter).
  - No accepta pseudoinstruccions (li, mv, j, call, ret…) ni directives (.equ, .data…): cal escriure les instruccions base.
- Al final del tema veurem com es codifica cada instrucció en 32 bits (codi màquina).

1. Introducció (III): Sobre la màquina RISCV-32I-simple

- L'arquitectura RISCV està pensada per a optimitzar
  - Circuiteria
  - Codi
  - Amb introducció de bits ocults (0's obviats)
  - Amb desplaçaments x2 o x4 en algunes instruccions per incrementar rang d'adreces

- **La màquina RISCV-32I-simple està pensada per**
  - Aprendre de forma educativa l'arquitectura i programació de la màquina RISCV-32I
- Per això,...
  - S'agafa la codificació de dades i instruccions de 32 bits.
  - Com a introducció al RISC-V simplifica algunes codificacions del codi màquina.
  - Com a iniciació a la creació d'algorismes en assemblador acota el conjunt d'instruccions del RISCV-32I.

1. Introducció (IV): Instruction set

- El *instruction set* (*conjunt d'instruccions*) especifica les instruccions que es poden emprar i el seu format.
- Les instruccions que es consideren són del tipus:
  - Aritmètiques de doble registre.
  - Aritmètiques amb immediat (constant).
  - De cerca o guarda de dades a memòria
  - De salt condicional.
  - De salt incondicional.

2. Instruccions aritmètiques de doble registre

- Consten de codi d'operació i 3 operands, que són les 3 variables amb les que s'opera.
  - Els operands corresponen a registres.
  - Format genèric: ADD rd, rs1, rs2
    - ADD: Codi d'operació
    - rd: Registre destí. Correspon a un dels 32 registres de la unitat de procés.
    - rs1, rs2: Registres fonts. Corresponen a un dels 32 registres de la unitat de procés.
  - Realitza l'operació: rd ← rs1 + rs2

- Exemple 1:
  - La instrucció que calcula l'hora civil a partir de l'hora UTC es pot posar com:

```
ADD horaCivil, horaUTC, GMT    # alt nivell
```

Fent l'assignació a registres:

x10 ← hora Civil

x11 ← horaUTC (GMT + 0)

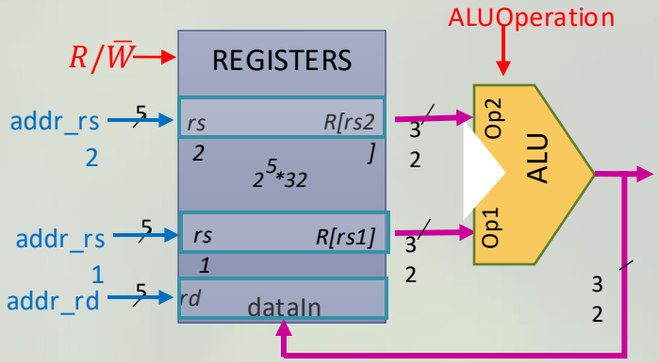
x12 ← GMT (Espanya GMT+1 o +2)

Queda la instrucció assemblador:

```
ADD x10, x11, x12          # assemblador
```

2. Instruccions aritmètiques de doble registre (II)

- En la UP s'executa l'operació



- El conjunt d'instruccions aritmètiques de doble registre de la màquina són:
  - ADD rd, rs1, rs2. Operació suma aritmètica: rd ← rs1 + rs2
  - SUB rd, rs1, rs2. Operació resta aritmètica: rd ← rs1 - rs2
  - XOR rd, rs1, rs2. Operació lògica or-exclusiva: rd ← rs1 ⊕ rs2
  - OR rd, rs1, rs2. Operació lògica or: rd ← rs1 | rs2
  - AND rd, rs1, rs2. Operació lògica and: rd ← rs1 & rs2
  - SLL rd, rs1, rs2. Desplaçament lògic a l'esquerra : rd ← rs1 << rs2 *(emplenant amb 0's)*
  - SRL rd, rs1, rs2. Desplaçament lògic a la dreta : rd ← rs1 >> rs2 *(emplenant amb 0's)*
  - SRA rd, rs1, rs2. Desplaçament aritmètic a la dreta : rd ← rs1 >> rs2 (*amb extensió de signe)*

2. Instruccions aritmètiques de doble registre (III): Desdoblament d'instruccions complexes

- Observar que cada instrucció en assemblador és una instrucció atòmica en el processador, que consta de:
  - 1 operació, indicada pel codi d'operació
  - Pot tenir un registre destí
  - 1 o 2 operands en registre. Per exemple, en instruccions aritmètiques de doble registre hi ha 2 operands.
- Les instruccions complexes s'han de dividir en instruccions assemblador.
- Exemple 2. Sigui la instrucció

$$z = (a+b)-(c+d)$$

Pas 1: Passant a operacions atòmiques

temp1 ← a+b

temp2 ← c+d

z ← temp1-temp2

Assignar registres

z = x10

a = x11

b = x12

c = x13

d = x14

temp1 = x20

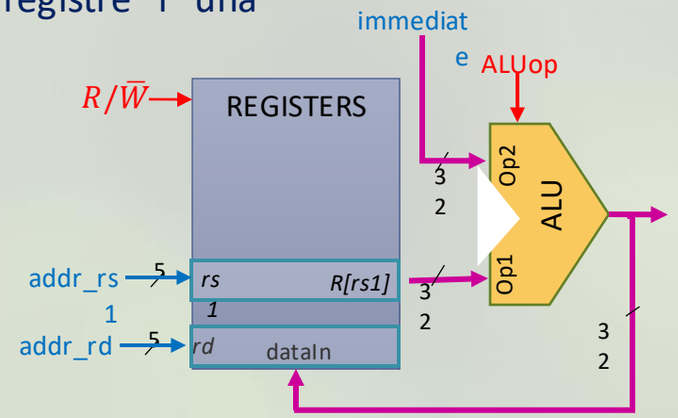
temp2 = x21

Pas 2: Codificar

- `ADD x20, x11, x12`
- `ADD x21, x13, x14`
- `SUB  x10, x20, x21`

3. Instruccions aritmètiques amb immediat

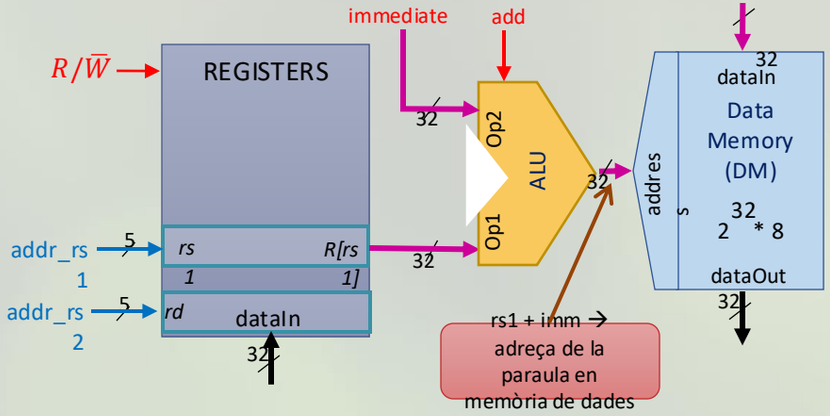
- Instrucció aritmètica que opera amb un registre i una constant.
- Format genèric: ADDI rd, rs1, imm
- Realitza l'operació: rd ← rs1 + imm
- La constant:
  - 12 bits, sempre amb extensió de signe:
    - $-2^{11} \leq imm \leq 2^{11} - 1$ (de −2048 a 2047)
  - En els desplaçaments: 5 bits (de 0 a 31).



- Repertori d'instruccions:
  - ADDI rd, rs1, imm. Operació suma aritmètica: rd ← rs1 + imm
  - XORI rd, rs1, imm. Operació lògica or-exclusiva: rd ← rs1 ⊕ imm
  - ORI rd, rs1, imm. Operació lògica or: rd ← rs1 | imm
  - ANDI rd, rs1, imm. Operació lògica and: rd ← rs1 & imm
  - SLLI rd, rs1, imm. Desplaçament lògic a l'esquerra : rd ← rs1 << imm *(emplenant amb 0's)*
  - SRLI rd, rs1, imm. Desplaçament lògic a la dreta : rd ← rs1 >> imm *(emplenant amb 0's)*
  - SRAI rd, rs1, imm. Desplaçament aritmètic a la dreta : rd ← rs1 >> imm (*amb extensió de signe)*

4. Instruccions d'accés a memòria

- Les dades amb les que treballa la ALU estan en la memòria de dades.
- L'accés a la mèmoria de dades es realitza via el banc de registres. La ALU no té accés directe amb la memòria de dades.
- L'adreça a la que s'accedeix a cercar una dada a la memòria de dades la calcula la ALU.
- L'adreça es calcula...
  - Proporcionant una adreça base guardada en un registre.
  - I sumant-li un desplaçament (offset) que permet recórrer la memòria a partir de l'adreça base.



- El conjunt d'instruccions d'accés a memòria de la màquina són:
  - LW rd, imm(rs1) (o LB per a un byte). Carrega una paraula (o un byte) a registre: rd ← mem(rs1 + imm)
  - SW rs2, imm(rs1) (o SB per a un byte). Guarda una paraula (o un byte) a memòria: mem(rs1 + imm) ← rs2

Nota: imm té 12 bits (−2048 a 2047). LH i SH (mitja paraula) també són de RV32I, però el simulador no les té.

4. Instruccions d'accés a memòria (II): LB, SB

Cas 1: es treballa amb paraules de 8 bits

- Exemple 3a. (Seguint amb l'exemple 1) Calcular l'hora civil a partir de l'hora UTC.

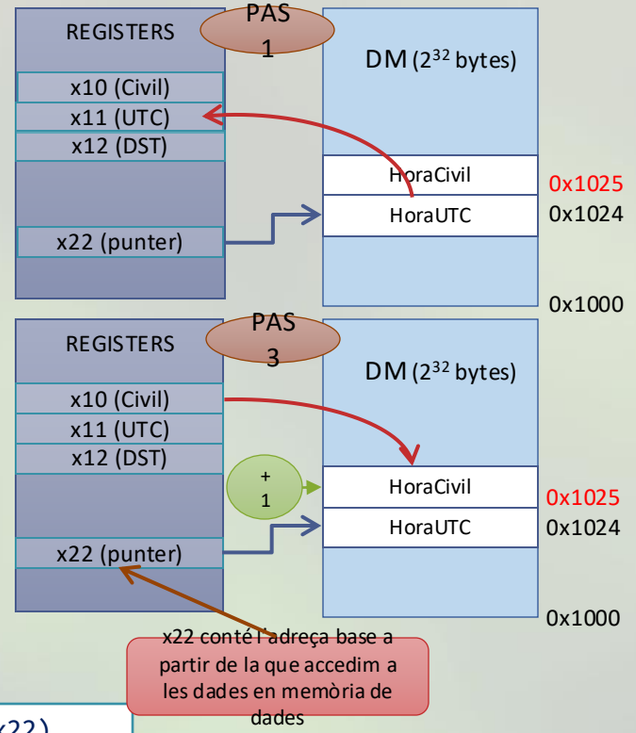
  Hora civil i UTC es troben en memòria
- Supòsits:
  - Les hores UTC i civil es guarden en posicions successives de memòria.
  - Les dades en memòria ocupen 1 byte.
  - El registre x22 conté l'adreça base de memòria (que apunta a l'hora UTC).
  - Els registres que s'usen són:
    - Registre temporal HoraCivil → x10
    - Registre temporal HoraUTC → x11
    - Registre temporal DST → x12
- La seqüència d'operacions és:

Anar a cercar UTC de memòria:

Operar: fer la suma HoraCivil ← HoraUTC+DST:

Guardar resultat:

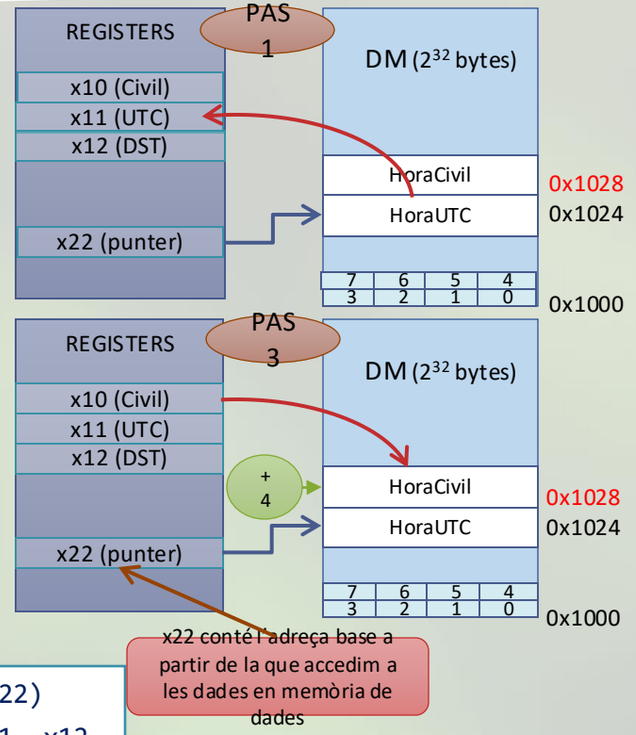
```
1: LB x11, 0(x22)
2: ADD x10, x11, x12
3: SB  x10, 1(x22)
```



4. Instruccions d'accés a memòria (III): LW, SW

Cas 2: es treballa amb paraules de 32 bits (RISC-V)

- Exemple 3b. Exemple 3a, però ara les dades són de tipus paraula (32 bits)
- Supòsits:
  - Les hores UTC i civil es guarden en posicions successives de memòria.
  - Les dades en memòria ocupen 4 bytes.
  - El registre x22 conté l'adreça base de memòria (que apunta a l'hora UTC)
  - Els registres que s'usen són:
    - Registre temporal HoraCivil → x10
    - Registre temporal HoraUTC → x11
    - Registre temporal DST → x12

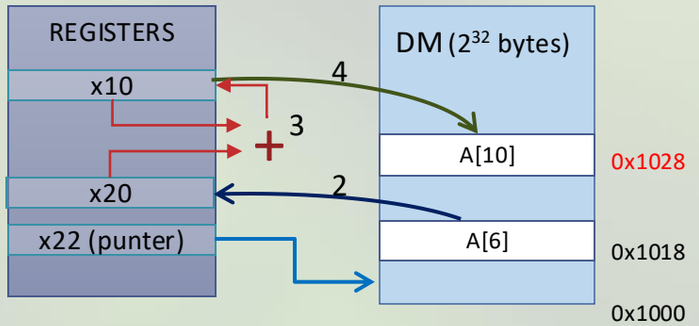


- La seqüència d'operacions és:

Anar a cercar UTC de memòria:	1: LW x11, 0(x22)
Operar: fer la suma HoraCivil ← HoraUTC+DST:	2: ADD x10, x11, x12
Guardar resultat:	3: SW x10, 4(x22)

4. Instruccions d'accés a memòria (IV): LW, SW

- Exemple 4. Sigui un array A guardat en memòria.
  Realitzar l'operació **A[10] = h + A[6]**.
  - L'array està guardat en memòria a partir de l'adreça 0x1000 i apuntat pel registre x22
  - x10 és un registre temporal que guarda h i el resultat.
- Plantejament:
  - Dades:
    - A[10], A[6]. En memòria.
    - Punter a memòria: x22
    - Registre x10 com a temporal per **h** i el resultat.
    - **h**: registre x10
- La seqüència d'operacions és:



Pseudocodi

```
x10 ← h
x20 ← mem[x22+4*6]
x10 ← x20 + x10
Mem[x22+4*10] ← x10
```

Codi

```
1: ADDI x10, x0, 5
2: LW x20, 24(x22)
3: ADD x10, x20, x10
4: SW  x10, 40(x22)
```